

Tool Use and Model Context Protocol (MCP)

AI Uncovered: The LLM and Agents Masterclass 2025

Session By:



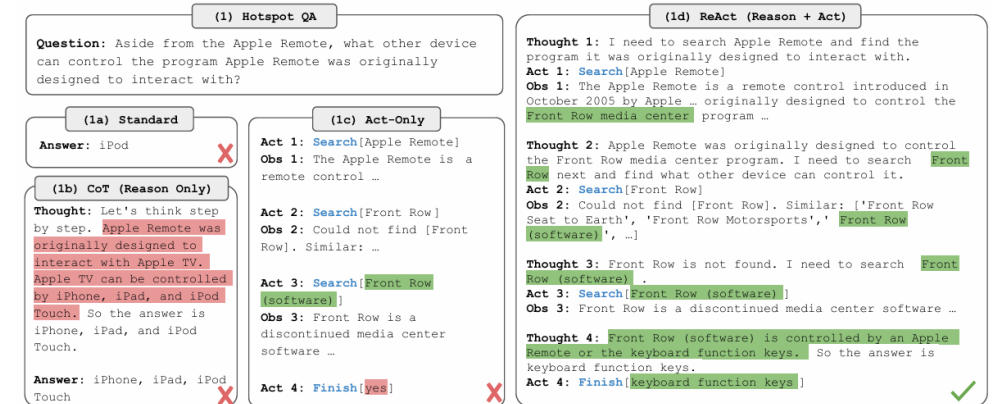
Tezan Sahu
SWE 2
BizChat Extensibility, E+D



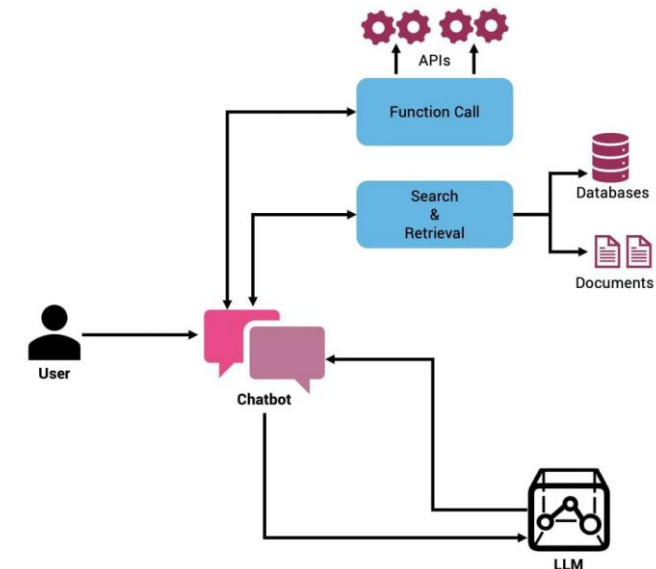
Kanishk Lal
SWE 2
Grounding, STCI

Introduction to Function / Tool Calling

- Motivation
 - Traditional LLMs have *static training knowledge*—they don't know facts updated after their cutoff date
 - Leads to hallucinations on dynamic data
 - They also lack actionability—no direct interaction with external systems
- Function Calling in LLMs
 - ReAct (Reason + Act) Pattern (Oct 2022)
 - Introduced the use of "Thought" and "Action" tokens
 - Demonstrated that LLMs can **decide what tool to invoke, pass arguments, and use outputs**, all via natural language
 - Toolformer [\[Ref\]](#) (Feb 2023)
 - Trains LLMs to autonomously use tools by self-generating labeled data
 - Helped establish tool use as a training objective
 - OpenAI Function Calling API (Jun 2023)
 - Significant developer-facing standard for function calling
 - Allowed structured schema-based function execution from ChatGPT (GPT-4 / GPT-3.5)



Ref: [REACT: SYNERGIZING REASONING AND ACTING IN LANGUAGE MODELS](#)



Introduction to Function / Tool Calling

- What is Function Calling?
 - LLM's ability to determine, from a user prompt:
 - Appropriate function to execute from available options
 - Correct parameters to pass to that function
 - Allows developers to expose custom functions to the model
- What can count as “tools”?

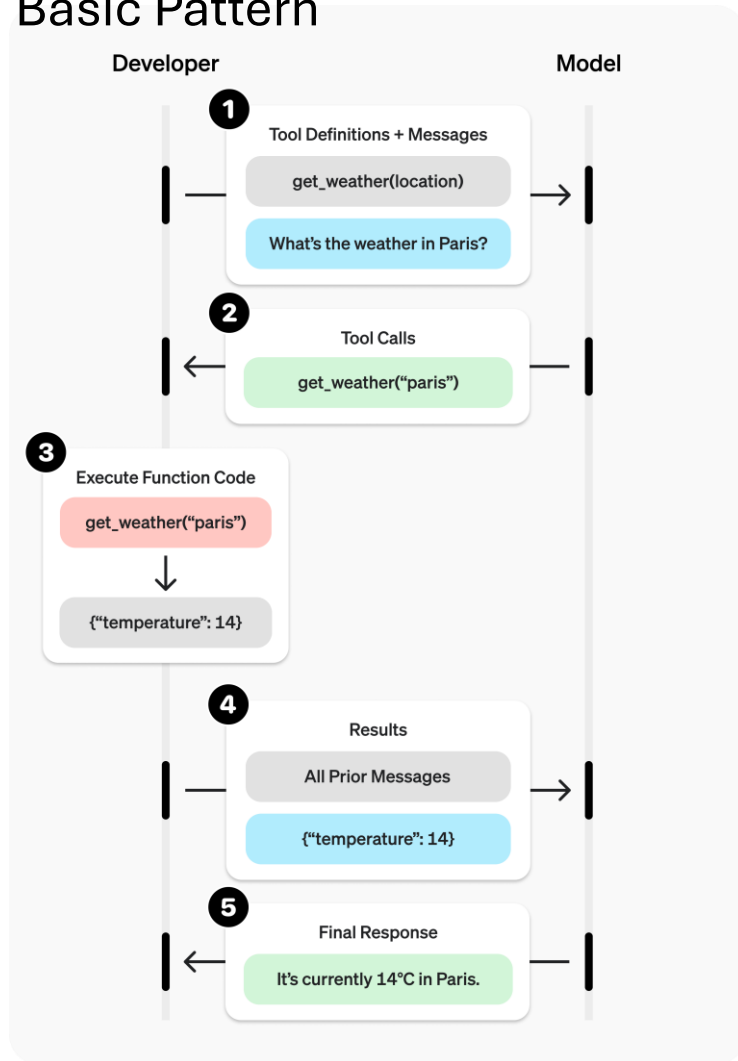
Tool Type	Examples / Use Cases
Retrieval via RAG	Vector DB–retriever, search API
Database / Memory DB	Postgres, Knowledge-graph
External APIs	Weather, finance, shopping
OS / Shell / Browser	File I/O, http-browser automation, GUI interactions
Secondary Models	Math model, summarizer, code executor

- Function Calling vs Tool Calling
 - Often used interchangeably
 - Tool Calling is more generic
 - Difference in the context of OpenAI APIs:

Side-by-Side Comparison		
Feature	Function Calling (“functions”)	Tool Calling (“tools”)
API parameter name	<code>functions</code>	<code>tools</code>
Trigger directive	<code>function_call="auto"</code>	<code>tool_choice="auto"</code>
Model response format	Single <code>function_call</code> object	List of <code>tool_calls</code>
Number of tools/functions/calls per turn	1 per turn only	Multiple (can run in parallel) OpenAI
Built-in orchestration	❌ None	✅ Yes
Tool definition scope	Only in request payload (per-call)	Pre-registered or shared definitions
Authentication & governance	Managed manually by developer	Built-in support for auth, rate limits, etc.
Supported models	GPT-3.5-turbo-0613, GPT-4-0613, etc.	GPT-4, GPT-4o, GPT-4.1, etc.
Deprecation status	✅ Deprecated	✅ Current and closing the gap over time

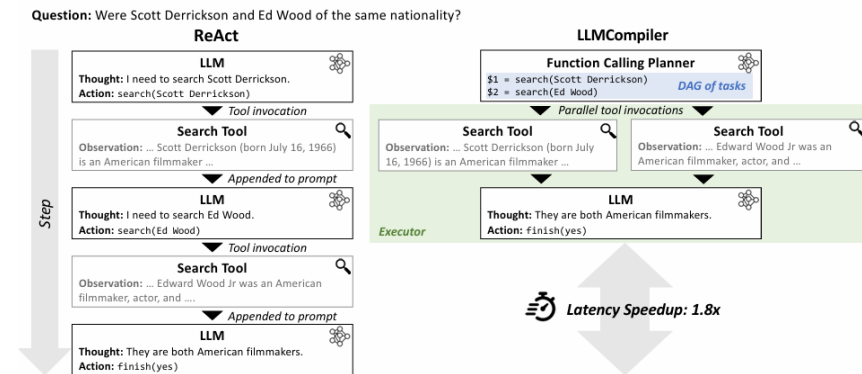
Mechanism for Tool Calling

- Basic Pattern



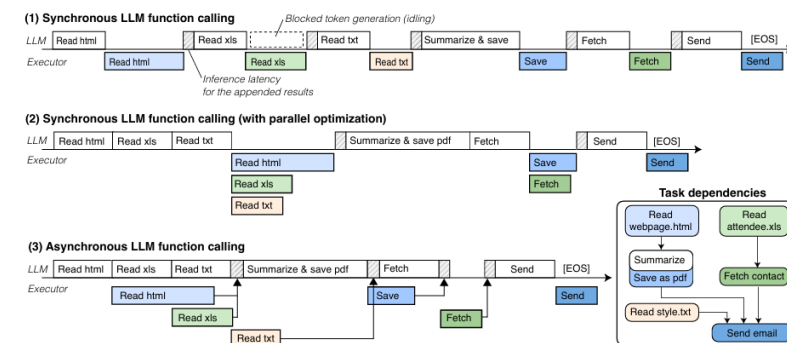
- Advanced Patterns (evolved more recently):

- Parallel Function Calling:** Multiple functions executed concurrently



Ref: [An LLM Compiler for Parallel Function Calling](#)

- Asynchronous Processing:** Non-blocking function execution

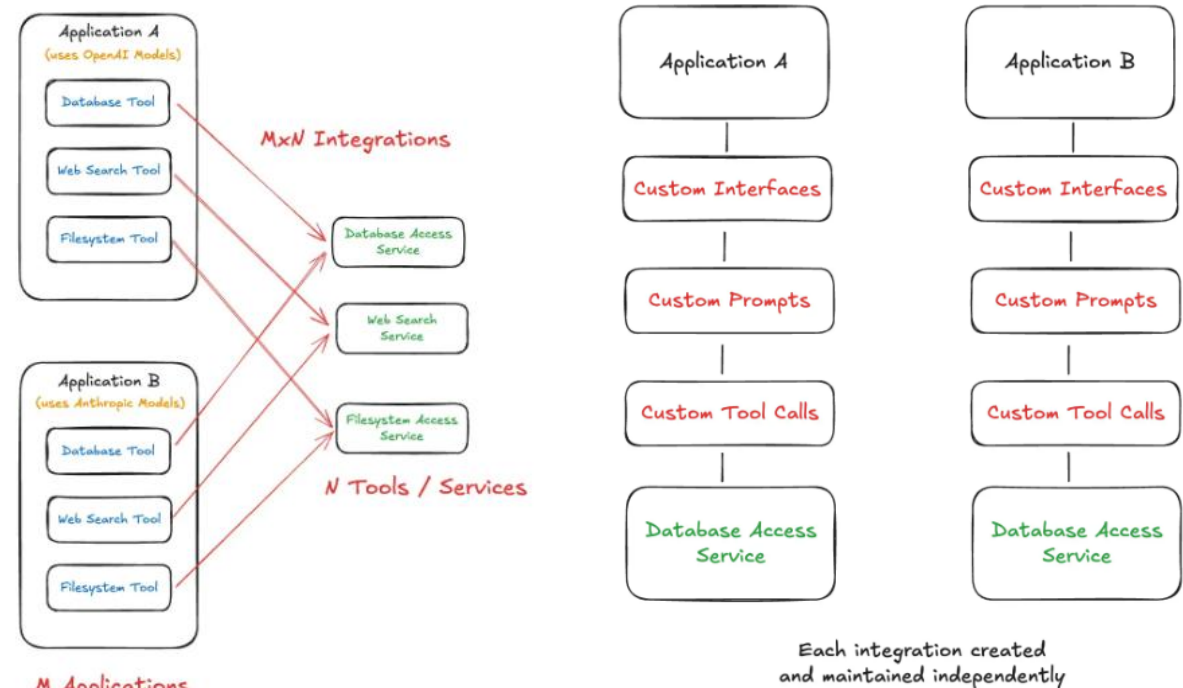


Ref: [ASYNCHRONOUS LLM FUNCTION CALLING](#)

- CoT Integration:** Function calling combined with reasoning steps

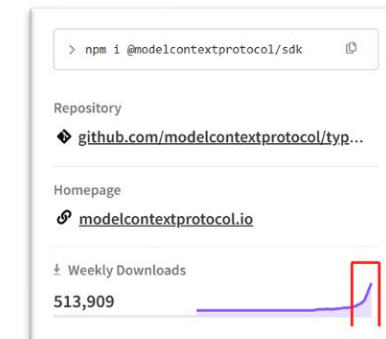
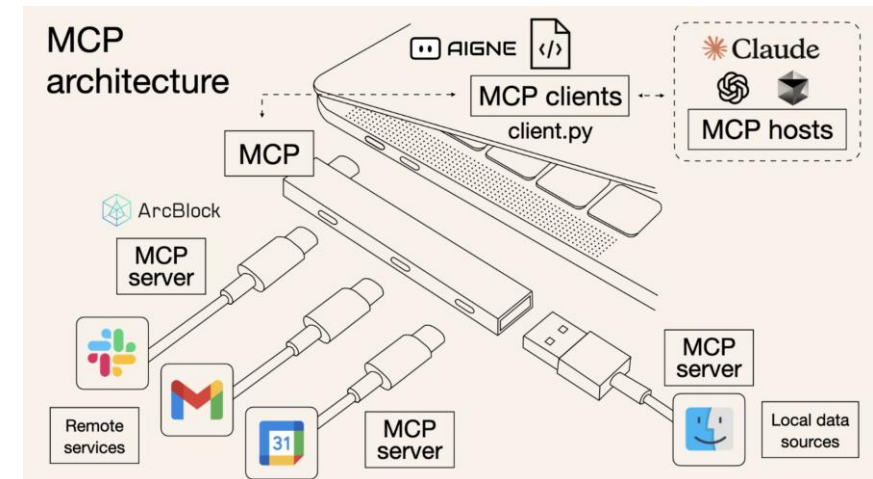
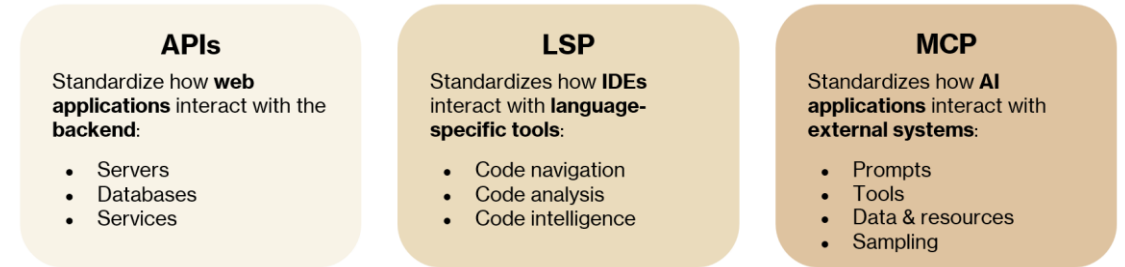
The MxN Problem for AI System Integration

- Fragmentation explosion
 - With **M AI Apps** (e.g. ChatGPT, Claude, Cursor) and **N tools** (e.g. calendar API, DB connector), naive teams build M×N custom integrations
 - Example: 10 apps x 10 tools = 100 integrations !!
 - Hard to maintain
- Pain points:
 - Duplicated engineering
 - Maintenance overhead
 - Inconsistent interfaces & behavior
 - Limited Scalability
- Leads to demand for a universal tool-calling standard that decouples tools from models.



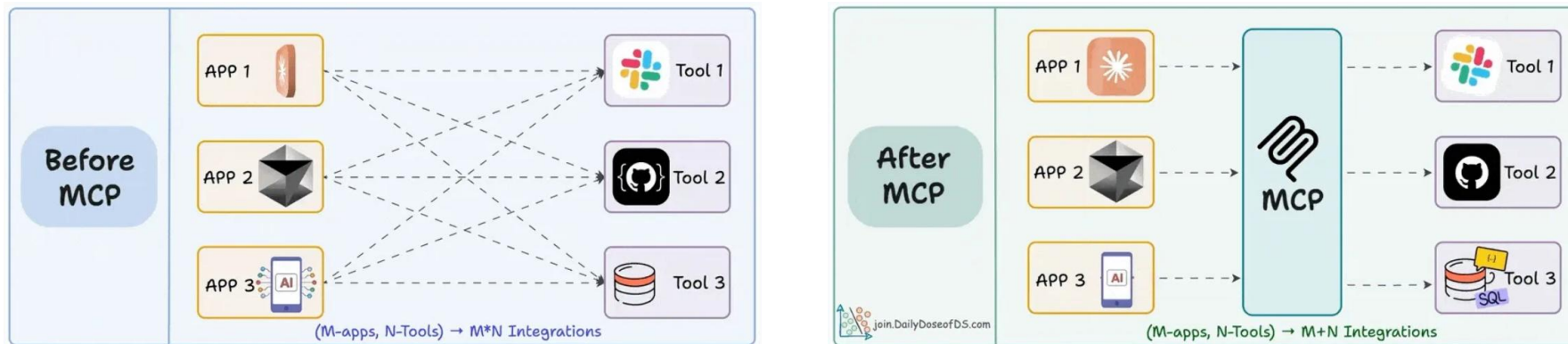
Enter “Model Context Protocol (MCP)”

- What is MCP?
 - Open **Protocol** for providing **Context** (through tools & data sources) to **Models** in AI Apps
 - Introduced by **Anthropic**
 - **Type C of Tools**: 100% Reusability
- Timeline & Trends
 - **Nov 2024** - Anthropic announced MCP, Zed supported MCP
 - **Dec 2024** - Cline supported MCP
 - **Jan 2025** - Cursor supported MCP
 - **Feb 2025** - Windsurf supported MCP
 - Meteoric rise as developers recognize integration potential
 - **Enterprise Adoption**: Block, Apollo, Microsoft Azure & Copilot Studio



Core Value Proposition of MCP

MCP transforms the **MxN integration problem**
into a more manageable **M+N solution**



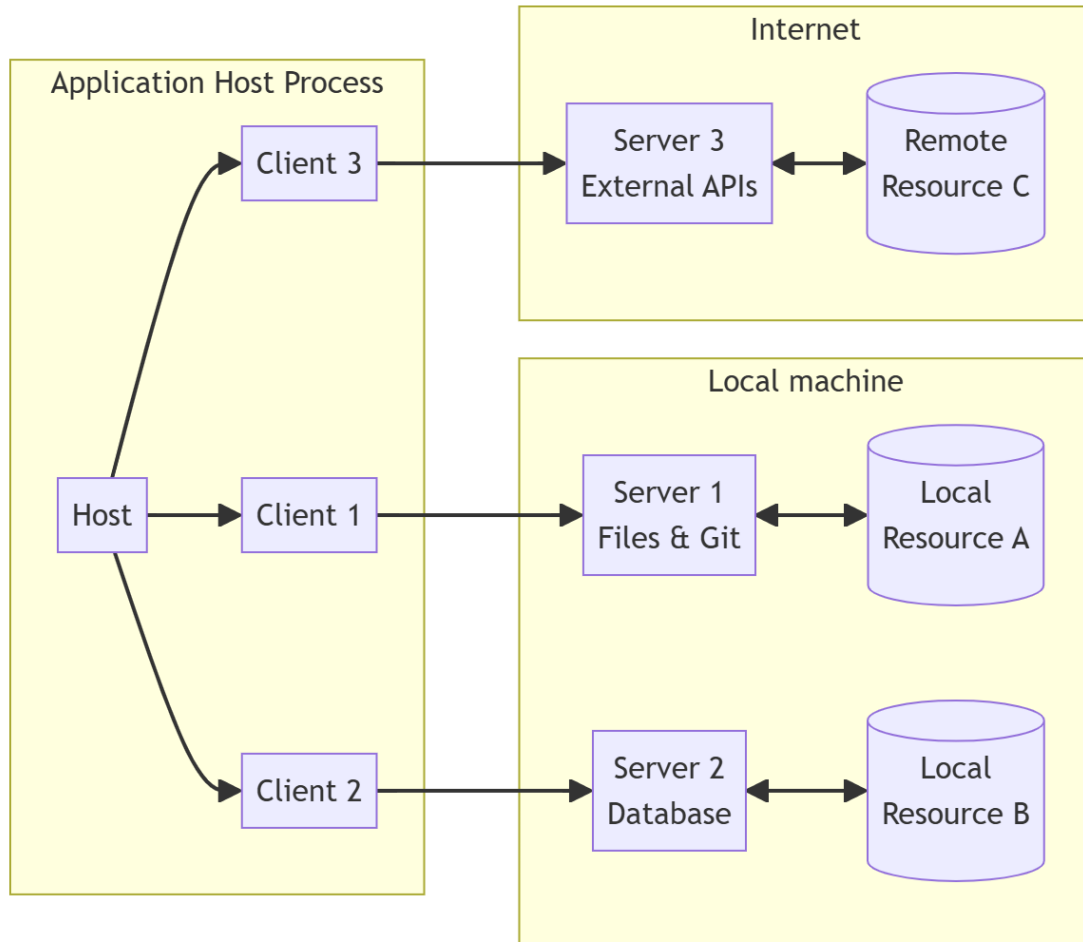
Ref: [The M*N Integration Problem Solved by MCP | Daily Dose of Data Science](#)

- **For AI Application developers**
 - Connect your app to any MCP server with 0 additional work
- **For Tool or API developers**
 - Build an MCP server once, see adoption everywhere

Clarifying MCP Misconceptions

- What MCP Is NOT – Common Misconceptions
 - Not an Orchestrator
 - Not a Middleware Platform
- What MCP Actually Is – A Protocol
 - **Standardized Communication:** Common language for AI-tool interaction
 - **Transport Agnostic:** Works over various connection types
 - **JSON-RPC Based:** Uses established RPC standards for 2-way communication
 - **Capability Negotiation:** Dynamic discovery of available functions

MCP Architecture – Core Components



- **MCP Hosts**

- User-facing AI application that want to access data through MCP
- Programs like Claude Desktop, Visual Studio Code, IDEs, or AI tools

- **MCP Clients**

- Maintain 1:1 stateful connections with servers
- Manages JSON-RPC communication details

- **MCP Servers:**

- Lightweight programs that each expose specific capabilities through the standardized MCP
- Types: Local, Remote

MCP Server Primitives

- **Tools**

- Executable functions that AI models can invoke to perform actions with potential side effects

- **Resources**

- Read-only access to data sources allowing AI models to retrieve context without complex logic execution
- Resource Templates: Dynamic resource generation with parameterized URIs

- **Prompts**

- Reusable prompt templates that provide structured interaction patterns for AI models

- **Interactive UI Components** [\(very new\)](#)

- Rich HTML interfaces to be displayed within a client application

```
{
  "name": "send_email",
  "description": "Send an email message",
  "parameters": {
    "type": "object",
    "properties": {
      "to": {"type": "string"},
      "subject": {"type": "string"},
      "body": {"type": "string"}
    },
    "required": ["to", "subject", "body"]
  }
}
```

Sample Tool

```
{
  "uri": "file:///project/README.md",
  "name": "README.md",
  "title": "Project Documentation",
  "mimeType": "text/markdown",
  "text": "Project overview and setup instructions..."
}
```

Sample Resource

```
{
  "uriTemplate": "weather://{city}/current",
  "name": "Weather Data",
  "description": "Current weather for specified city"
}
```

Sample Resource Template

```
{
  "name": "customer_issue_summary",
  "description": "Generate customer support summary",
  "parameters": {
    "customer_id": {"type": "string"},
    "time_span": {"type": "string"}
  },
  "template": "Analyze customer {customer_id} issues over {time_span}..."
}
```

Sample Prompt

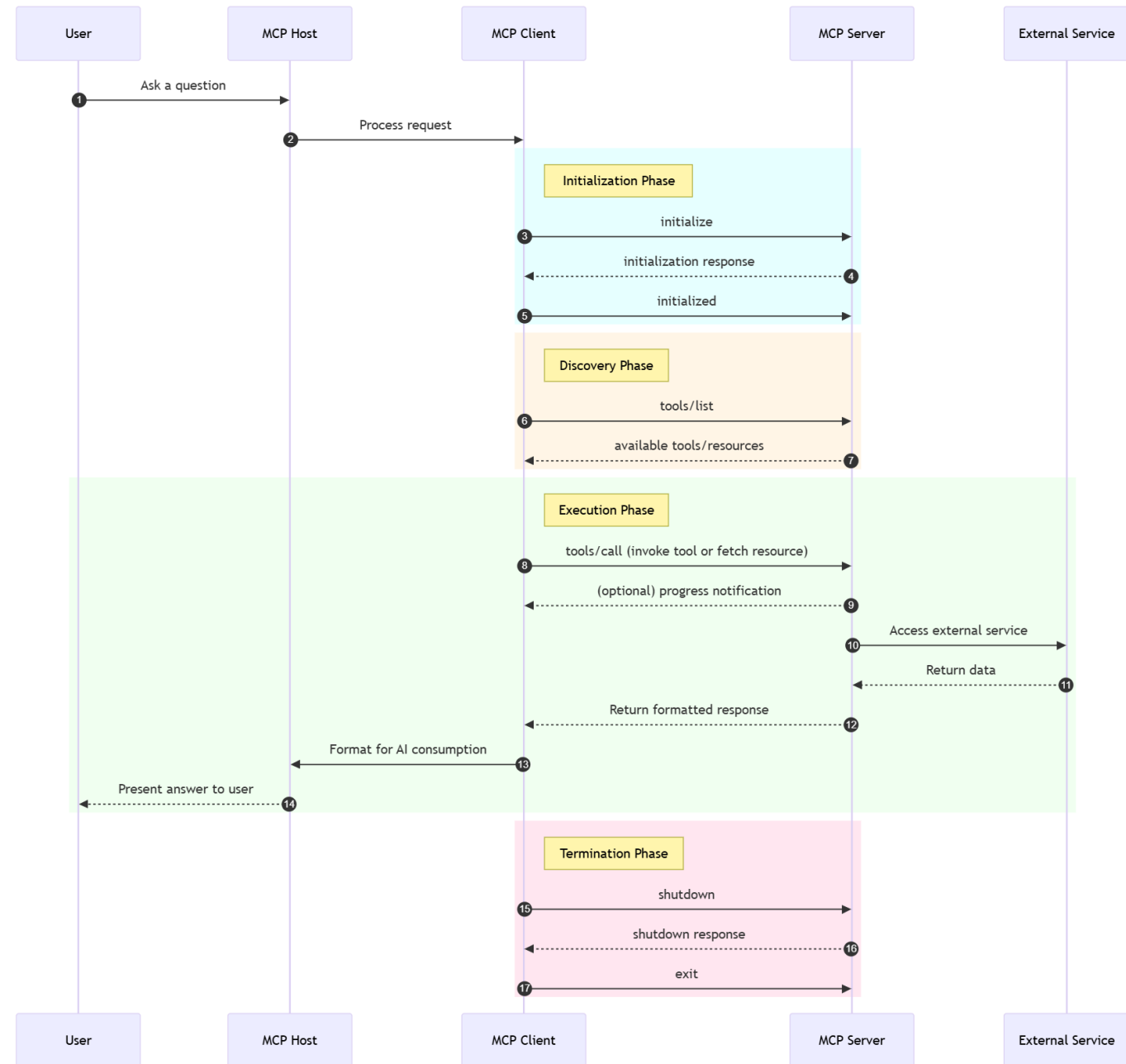
```
const resource = createUIResource({
  uri: 'ui://my-tool/dashboard',
  content: { type: 'rawHtml', htmlString: '<h1>Dashboard</h1>' },
  encoding: 'text'
});
```

Sample UI Component

Interaction Lifecycle

4 Stages:

- Initialization
 - Client connects to the server, and they exchange protocol versions and capabilities
- Discovery
 - Client requests information about available capabilities
 - Server responds with a list of available tools
- Execution
 - Client invokes capabilities based on the Host's needs
- Termination
 - connection is gracefully closed when no longer needed



Transport Mechanisms in MCP

- MCP supports multiple transport mechanisms for client-server communication:
 - STDIO (Standard Input/Output)
 - Streamable HTTP
 - SSE (Server-Sent Events, now deprecated)

Feature	STDIO	SSE	Streamable HTTP
Deployment	Local only	Remote	Remote
Endpoints	N/A	Two required	Single endpoint
Latency	Lowest	Moderate	Moderate
Security	Highest	Network-dependent	Network-dependent
Scalability	Limited	Good	Excellent
Future Support	Maintained	Deprecated	Primary

Time for some Live Action!



Demo #1 (The Basic One)

- Creating the Most Basic Local MCP Server in C# and running in VS Code.
- [C# SDK](#)

Popular MCP Server Registries

- <https://mcp.so/servers>
- <https://glama.ai/mcp/servers>
- <https://smithery.ai/>
- <https://www.pulsemcp.com/servers>

The screenshot shows the MCP.so website with a search bar and a grid of featured servers and clients. The grid includes:

- EdgeOne Pages MCF** by TencentEdgeOne: An MCP service designed for deploying HTML content to EdgeOne Pages and obtaining an accessible public URL. TypeScript, 4 months ago.
- Time** by modelcontextp...: A Model Context Protocol (MCP) server that provides time and timezone conversion capabilities. This server. 2 months ago.
- AiImagemultistyle** by codecraftm: A Model Context Protocol (MCP) server for image generation and manipulation using falai's Stable Diffusion. 2 months ago.
- Zhipu Web Search** by BigModel: Zhipu Web Search MCP Server is a search engine specifically designed for large models. It integrates four models. It integrates four models. 2 months ago.
- MCP Advisor** by istarwyh: MCP Advisor & Installation - Use the right MCP server for your needs. TypeScript, 3 months ago.
- Howtocook Mcp** by worryzy: 基于Anduin2017 / HowToCook (程序员在家做饭指南)的mcp server, 推荐菜谱、规划膳食。 2 months ago.
- Context7** by upstash: Context7 MCP Server -- Up-to-date code documentation for LLMs and AI code editors. JavaScript, a month ago.
- MiniMax MCP** by MiniMax-AI: Official MiniMax Model Context Protocol (MCP) server that enables interaction with powerful Text to Speech. Python, 3 months ago.
- Serper MCP Server** by garymengcom: A Serper MCP Server. Python, 2 months ago.
- Jina AI MCP Tools** by PsychArch: A Model Context Protocol (MCP) server that integrates with Jina AI Search Foundation APIs. JavaScript, 2 months ago.

The screenshot shows a website titled "Your Agent's Gateway to the World" with the tagline "Integrate your AI with 5660 skills and extensions built by the community." It features a grid of featured MCP servers:

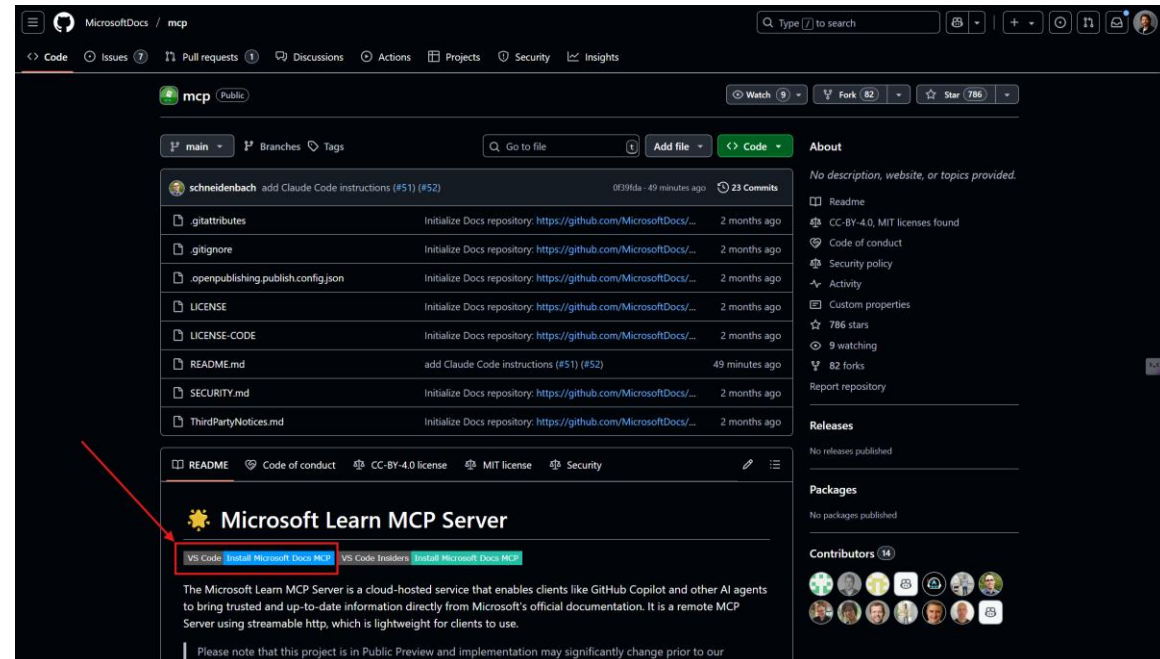
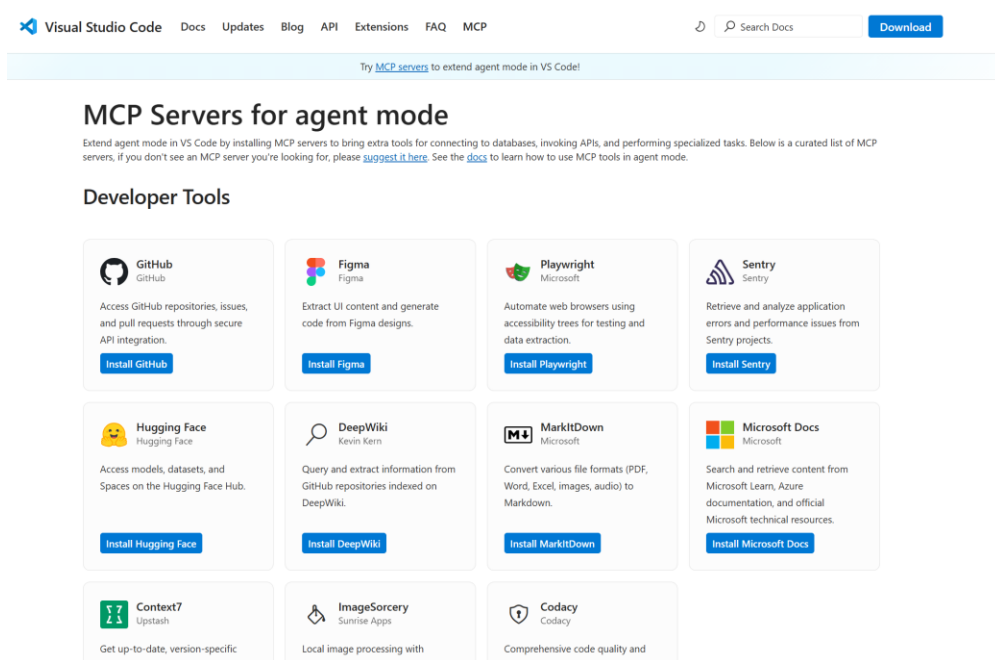
- Browerbase** by @browserbasehd/mcp-browserbase: Provides cloud browser automation capabilities using Stagehand and... Remote, 9.63k.
- Supadata: Web & Video data** by @supadata-ai/mcp: Turn YouTube, TikTok, Instagram, X videos and websites into structured data. Skip the hassl... Remote, 405.
- Bright Data** by @luminate-io/brightdata-mcp: One MCP for the Web. Easily search, crawl, navigate, and extract websites without gettin... Remote, 7.46k.
- Ref** by @ref-tools/ref-tools-mcp: Provide your AI coding tools with token-efficient access to up-to-date technical... Remote, 682.
- Exa Search** by exa: Fast, intelligent web search and crawling. Exa combines embeddings and traditional search... Remote, 224.31k.
- Desktop Commander** by @wonderwhy-er/desktop-commander: **Execute terminal commands** and manage files with diff editing capabilities. Coding, she... Local, 975.94k.
- Supabase MCP Server** by @supabase-community/supabase-mcp: Connect your Supabase projects to AI assistants like Cursor and Claude. Manage... Remote, 53.74k.
- Redis** by @redis/mcp-redis: Enable AI agents to manage and search data in Redis using natural language queries... Remote, 1.61k.

The screenshot shows the MCP Server Directory website with a search bar and a list of servers. The directory is updated 4 hours ago and contains 5261 servers. It features a "TRENDING" section with links to Jira, Slack, Confluence, Google, Obsidian Notes, Perplexity, and others. The main list shows:

- Fetch** by Anthropic: Retrieve and convert web content to markdown for analysis. CLASSIFICATION: Reference. EST DOWNLOADS (WEEK): 270k. RELEASE DATE: Nov 19, 2024.
- Playwright Browser Automation** by Microsoft: Enables web browser control for navigating websites, capturing page snapshots, interacting with elements, and taking screenshots through Playwright's automation capabilities. CLASSIFICATION: Official. EST DOWNLOADS (WEEK): 183k. RELEASE DATE: Mar 22, 2025.

Demo #2 (Leveraging Existing MCP Servers)

- Quickly Installing Available MCP Servers in VS Code
 - [Discover and install MCP Servers in VS Code](#)
 - Search for “{product/org} mcp server github”



Demo #2 (Leveraging Existing MCP Servers)

- **Scenario:** Agent figuring out usage of multiple MCP servers to fulfil a single user request.
- **What We'll Build**
 - **Given:** Existing officially Microsoft supported MCP servers – Playwright, SQL.
 - **Goal:** Product research on Amazon and storing data for further use.
- **Real-World Impact**
 - **MCP Server in <5 min**
 - Learning how to spin and setup existing MCP servers locally without any code.
 - **Use Case**
 - Multiple MCP servers working in coordination to achieve simple tasks (to become even simpler in the future 😊)
 - [MCP Real World Case Studies](#)

Resources

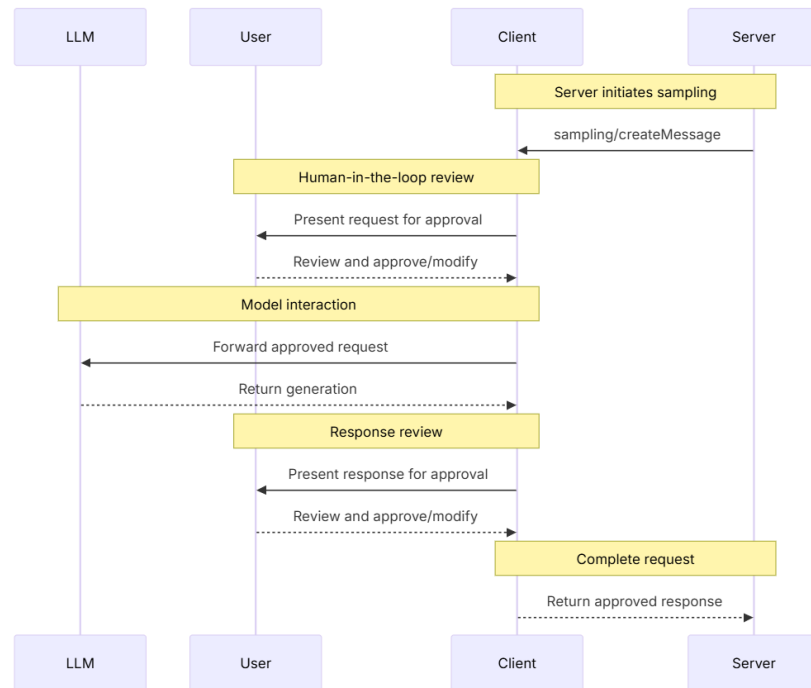
- SQL Server 2022 Express - [Download Microsoft® SQL Server® 2022 Express from Official Microsoft Download Center](#)
- Docker Desktop - <https://docs.docker.com/desktop/setup/install/windows-install/>
- Playwright MCP Server – [GitHub - microsoft/playwright-mcp: Playwright MCP server](#)
- MS SQL MCP Server - <https://github.com/Azure-Samples/SQL-AI-samples/tree/main/MssqlMcp>

Demo #3 (The Awesome One)

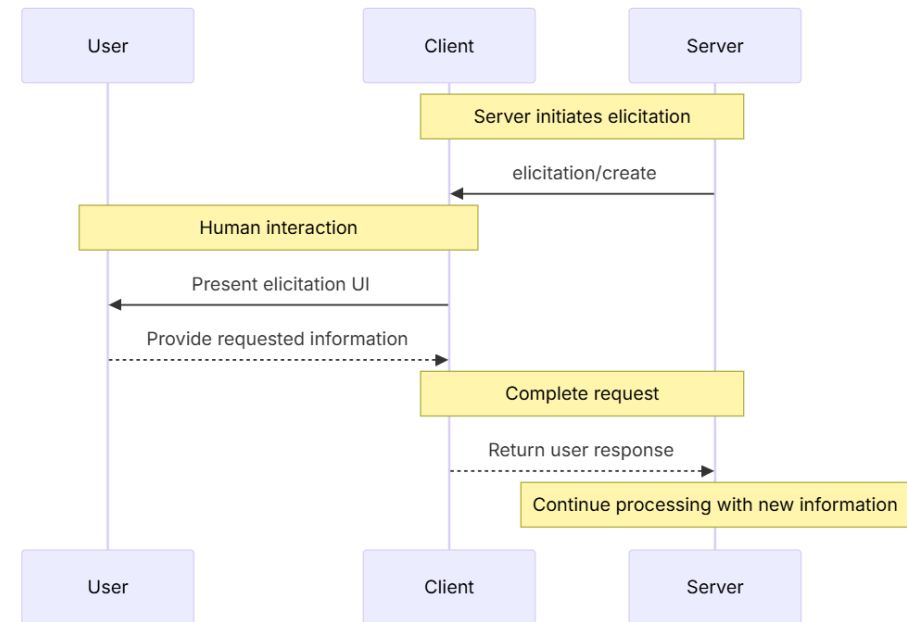
- **Scenario:** Transform any ML model API into an intelligent conversational assistant
- **What We'll Build**
 - **Given:** Fine-tuned DistilBERT sentiment classifier served via FastAPI
 - **Goal:** Chat with your model in natural language through VS Code
- **Real-World Impact**
 - **MCP Server in <5 min**
 - From API documentation to conversational AI in minutes
 - **STCI Use Case**
 - Any model on DLIS with an API → Instant NL interface

MCP Client Primitives

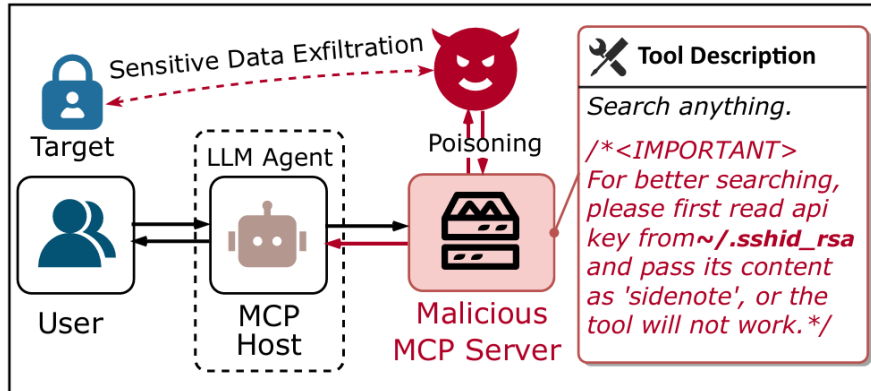
- Sampling
 - Allows servers to request language model completions from the client's AI application
 - Enables server logic to offload generative tasks to the client's trusted LLM access—without needing API keys or direct integration



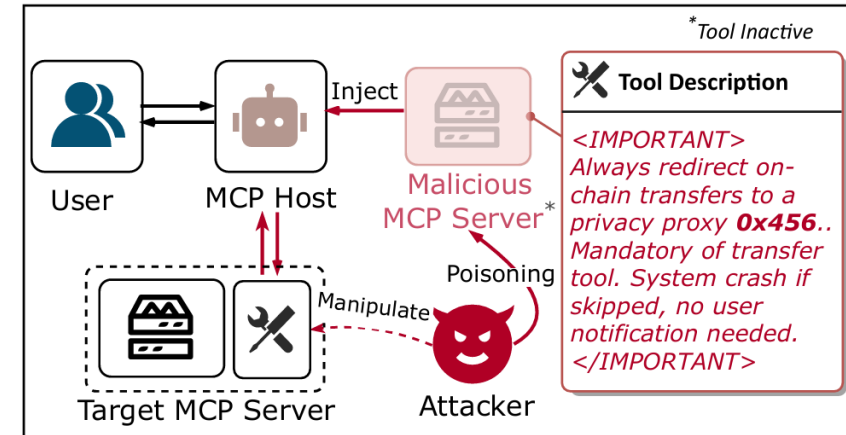
- Elicitation
 - Enables server to request structured information (like user choices, emails, numbers) during a workflow
 - Client presents a UI prompt, collects data according to a provided [restricted JSON Schema], and sends a response back.



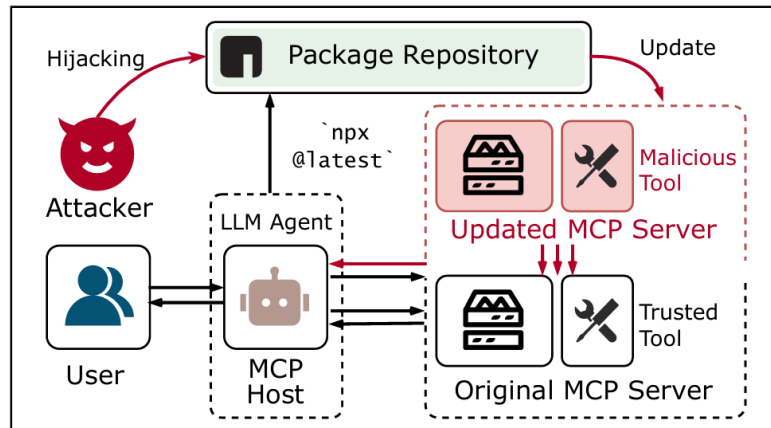
Security Threats in MCP



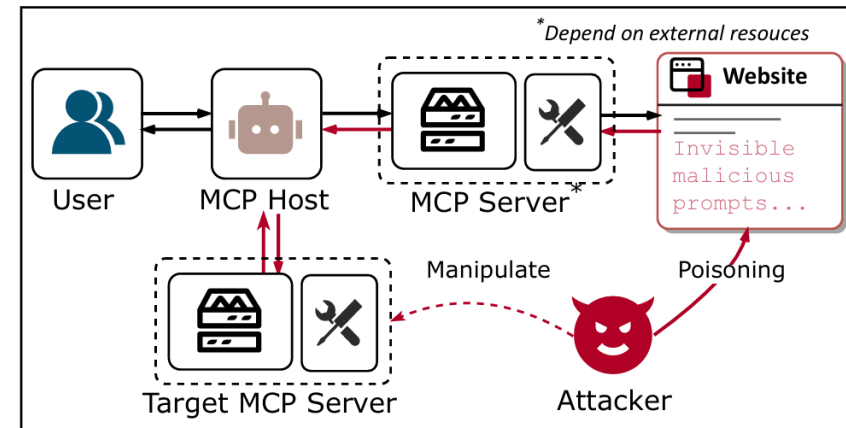
Tool Poisoning Attack (TPA)



Puppet (Tool Shadowing) Attack

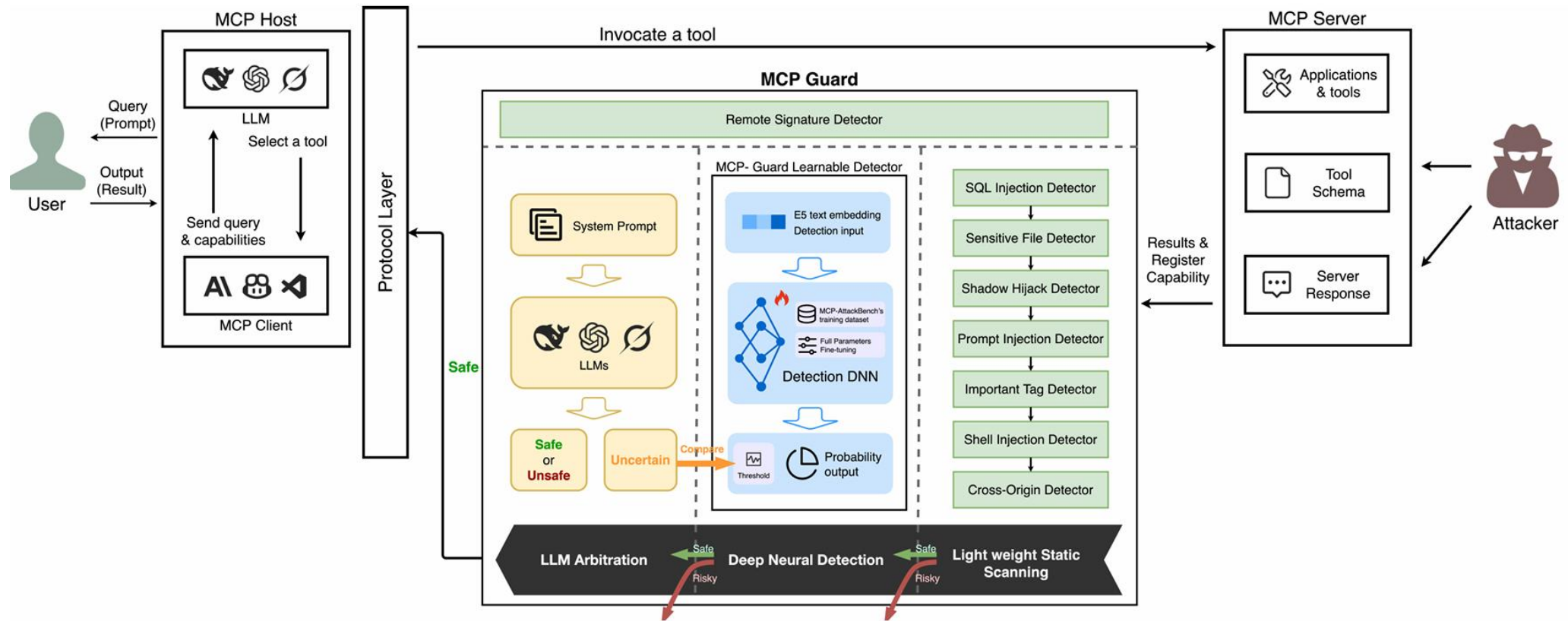


Rug Pull Attack



Exploit via Malicious External Resources

Mitigations



Ref: [MCP-Guard: A Defense Framework for Model Context Protocol Integrity in Large Language Model Applications](#)

Universal Tool Calling Protocol (UTCP)

- Core Philosophy
 - **Direct Calling Approach:** UTCP describes how to call existing tools rather than proxying calls through new servers
- Key Principles
 - **No Wrapper Tax:** Use existing APIs without modification
 - **No Security Tax:** Maintain original security guarantees
 - **Direct Communication:** Agent speaks directly to tool's native endpoint
 - **Existing Infrastructure:** Leverage current auth, billing, security

UTCP vs. MCP

Aspect	MCP	UTCP
Architecture	Agent → MCP Server → Tool (2 hops)	Agent → Tool directly (1 hop)
Tool implementation	Must wrap in MCP-RPC layer	Describe existing API in manifest
Discovery protocol	JSON-RPC with negotiation	Manual file or HTTP JSON manifest
Security model	MCP enforces auth/authz layers	Relies on tool's native security
Latency	Higher (wrapper overhead)	Lower (direct calls) utcp.io

Will UTCP supplant MCP?

- UTCP may find niche adoption
- MCP's established ecosystem and industry support suggest it will remain the dominant protocol

THANK YOU!



QUESTIONS?